



JS-3: Array & Object Methods

map · filter · reduce · find · some · every · rest/spread

A complete first-principles mini booklet — zero to advanced — with real-world examples, runnable code, interview questions, and tricky output-based traps. Built for MERN Stack trainers to teach from scratch.

MyAnatomy Teaching Series

Table of Contents

1. First Principles — What Even IS an Array Method?	3
2. The Callback Function — The One Idea Everything Depends On	4
3. map() — Transform Every Element	5
4. filter() — Keep Only What You Want	7
5. reduce() — Collapse an Array Into One Thing	9
6. find() & findIndex() — Get One Matching Element	12
7. some() & every() — Ask a Yes/No Question About the Array	13
8. Rest Parameter — Collecting Loose Values	15
9. Spread Operator — Unpacking Values	16
10. Rest vs Spread — Side by Side (the #1 confusion)	17
11. Chaining Methods — Real Mini-Projects	18
12. Comparison Cheat-Sheet — map vs forEach vs filter vs reduce	20
13. Interview Questions (Conceptual)	21
14. Tricky Output-Based Questions	23
15. Practice Problems for Students (Homework Set)	26

1. First Principles — What Even IS an Array Method?

1.1 Start from zero: what is an array?

An array is just an ordered list of values stored under one variable name, each with a numeric position (index) starting at 0.

```
const fruits = ["apple", "banana", "mango"];
// index:      0         1         2
```

1.2 Before ES5: how did we process arrays?

Before methods like map/filter existed, the only tool was a manual `for` loop:

```
const fruits = ["apple", "banana", "mango"];
const upper = [];
for (let i = 0; i < fruits.length; i++) {
  upper.push(fruits[i].toUpperCase());
}
console.log(upper); // ["APPLE", "BANANA", "MANGO"]
```

This works, but every time you write it you must correctly write the loop bounds, remember to create an empty array, remember to push into it, and remember not to go out of bounds. That's a lot of repeated boilerplate for a very common task: "do something to every item."

1.3 Why array methods were invented

The "why": JavaScript's designers noticed that 90% of loops over arrays fall into a small number of repeating patterns: transform every item, keep only some items, combine all items into one value, find one item, or check a yes/no condition. So instead of every developer re-writing the same `for` loop boilerplate forever, they built these patterns directly into the Array prototype as ready-made methods: `map`, `filter`, `reduce`, `find`, `some`, `every`. You just supply the one part that changes each time — the logic — as a function.

1.4 What "higher-order function" means (the formal term)

A **higher-order function** is any function that either (a) accepts another function as an argument, or (b) returns a function. `map`, `filter`, and `reduce` are all higher-order functions because you hand them a function to run on each element. This is the formal vocabulary word interviewers expect you to know.

1.5 Where these methods live

They are built into `Array.prototype`, which is why every array in JavaScript automatically has access to them — you never need to import anything.

```
console.log(typeof [].map); // "function"
console.log([].map === Array.prototype.map); // true
```

1.6 When to reach for which method (quick preview)

You want to...	Use
Transform every item into something else (same length out)	<code>map()</code>
Keep only items matching a condition (shorter or equal length out)	<code>filter()</code>
Combine all items into a single value (sum, object, string...)	<code>reduce()</code>

Get the first item matching a condition	<code>find()</code>
Check if AT LEAST ONE item matches a condition (true/false)	<code>some()</code>
Check if ALL items match a condition (true/false)	<code>every()</code>

2. The Callback Function — The One Idea Everything Depends On

2.1 First principle: functions are values in JavaScript

In JavaScript, a function is not a special, separate kind of thing — it is a value, just like a number or a string. This means you can:

- Store it in a variable
- Pass it as an argument into another function
- Return it from another function

```
function sayHi() {
  console.log("Hi!");
}
const greet = sayHi; // stored in a variable, just like a number
greet();              // "Hi!"
```

2.2 What is a "callback"?

A **callback function** is simply a function that you pass as an argument into another function, to be "called back" (invoked) by that other function at the right moment.

```
function processUser(name, callback) {
  console.log("Processing:", name);
  callback(name); // <-- the outer function decides WHEN to call it
}

processUser("Riya", function(n) {
  console.log(n + " has been processed!");
});
```

Key insight: When you write `array.map(fn)`, you are not calling `fn` yourself. You are handing `fn` to `map`, and `map` is the one that calls it — once for every element, automatically. This is exactly the same pattern as `processUser` above, just built into the language.

2.3 The three arguments every array-method callback receives

Whatever callback you pass to `map/filter/forEach/etc.`, JavaScript automatically calls it with THREE arguments (you can use as many or as few as you need):

```
array.map(function(element, index, array) {
  // element = the current item's value
  // index   = the current item's position (0, 1, 2...)
  // array   = the entire original array (rarely needed)
});
```

Beginners almost always only use `element`. That's fine and normal for 80% of cases. But knowing `index` exists will save you later (e.g. numbering a list, or comparing neighbors).

2.4 Arrow function shorthand (what you'll see in real code)

```
// Traditional function expression
numbers.map(function(n) { return n * 2; });

// Arrow function — same thing, shorter
numbers.map((n) => { return n * 2; });

// Arrow function — implicit return (no curly braces, no "return" keyword)
numbers.map(n => n * 2);
```

Common student mistake: Writing `n => { n * 2 }` with curly braces but forgetting the `return` keyword. With curly braces, JS assumes you'll write full statements and will NOT auto-return anything — the result is `undefined` for every element. Without curly braces, the expression after the arrow is automatically the return value. This single mistake causes an enormous number of "why is my array full of undefined" bugs.

3. map() — Transform Every Element

3.1 What it does, in one sentence

`map()` takes an array, runs your function on **every** element, and returns a **brand new array** of the same length, where each element is whatever your function returned for it.

3.2 Syntax

```
const newArray = originalArray.map((element, index, array) => {
  return transformedElement;
});
```

3.3 Building it from first principles (what map is doing under the hood)

If `map` didn't exist, you would write it yourself like this:

```
function myMap(arr, callback) {
  const result = [];           // ALWAYS a new array
  for (let i = 0; i < arr.length; i++) {
    result.push(callback(arr[i], i, arr)); // Step 3: ALWAYS keep the return value
  }
  return result;
}
```

This is genuinely very close to how the real `map` is implemented. If a student can write `myMap` from memory, they understand map completely.

3.4 Step-by-step trace (do this on the whiteboard)

```
const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2);
```

Iteration	element	index	callback returns	pushed into new array
1st	1	0	1*2 = 2	[2]
2nd	2	1	2*2 = 4	[2, 4]
3rd	3	2	3*2 = 6	[2, 4, 6]

```
console.log(doubled); // [2, 4, 6]
console.log(nums);    // [1, 2, 3] <-- UNCHANGED! map never mutates the original
```

3.5 Real-world example 1 — formatting API data for display

You fetch a list of users from an API. The backend gives you `first_name` and `last_name`, but your UI needs a single `fullName` string to display in a list. This is map's bread-and-butter job: reshape data from one form into another.

```
const usersFromAPI = [
  { first_name: "Aarav", last_name: "Sharma", age: 21 },
  { first_name: "Priya", last_name: "Verma", age: 23 }
];

const displayUsers = usersFromAPI.map(user => ({
  fullName: `${user.first_name} ${user.last_name}`,
  age: user.age
}));

console.log(displayUsers);
// [{ fullName: "Aarav Sharma", age: 21 }, { fullName: "Priya Verma", age: 23 }]
```

Syntax trap: When an arrow function's implicit return is an **object literal**, you must wrap it in parentheses: `user => ({ ... })`. Without the parentheses, JS thinks the `{` is the start of a function body, not an object, and you'll get a syntax error or `undefined`.

3.6 Real-world example 2 — rendering a list in React

This is the single most common real-world use of map for a MERN developer: turning an array of data into an array of JSX elements.

```
function ProductList({ products }) {
  return (
    <ul>
      {products.map(product => (
        <li key={product.id}>{product.name} - ₹{product.price}</li>
      ))}
    </ul>
  )
}
```

```
    </ul>
  );
}
```

Tell students: "Every time you've seen a list of cards or rows in a website, there's almost certainly a `.map()` behind it."

3.7 Real-world example 3 — unit conversion

```
const celsiusTemps = [0, 20, 37, 100];
const fahrenheitTemps = celsiusTemps.map(c => (c * 9/5) + 32);
console.log(fahrenheitTemps); // [32, 68, 98.6, 212]
```

3.8 map() vs forEach() — the question every student asks

	map()	forEach()
Returns	A new array	undefined
Use when	You need a transformed array to use later	You just want to "do something" for each item (e.g. log it, save to DB) and don't need a new array
Chainable?	Yes (returns array, so <code>.filter()</code> , <code>.map()</code> etc. can follow)	No — chaining after forEach is a bug (you'd be chaining onto <code>undefined</code>)

Classic beginner bug:

```
// WRONG — using map just to "loop", throwing away the result
numbers.map(n => console.log(n)); // works but is misleading/wasteful — use forEach

// WRONG — expecting forEach to give you a new array
const doubled = numbers.forEach(n => n * 2);
console.log(doubled); // undefined!! forEach never returns anything useful
```

Rule of thumb to teach: "If you use the word 'return' meaningfully and need the result → map. If you're just performing an action with no need for a result → forEach."

4. filter() — Keep Only What You Want

4.1 What it does, in one sentence

`filter()` runs your function (which must return `true` or `false`) on every element, and returns a **new array** containing only the elements for which your function returned `true`.

4.2 Syntax

```
const newArray = originalArray.filter((element, index, array) => {
  return true or false; // a "predicate" - a condition
});
```

The function you pass to filter is called a **predicate** — a function whose whole job is to answer a yes/no question about each item.

4.3 Building it from first principles

```
function myFilter(arr, predicate) {
  const result = [];
  for (let i = 0; i < arr.length; i++) {
    if (predicate(arr[i], i, arr)) { // Step 3: keep ONLY if true
      result.push(arr[i]); // note: push the ORIGINAL element, not the predicate's return value
    }
  }
  return result;
}
```

Compare this to map's Step 3 above: map always pushes the callback's *return value*; filter pushes the *original element* only when the callback's return value is truthy. This one-line difference is the entire distinction between the two methods — make students say it back to you.

4.4 Step-by-step trace

```
const nums = [4, 9, 15, 22, 7];
const evens = nums.filter(n => n % 2 === 0);
```

element	n % 2 === 0	kept?
4	true	✓ yes
9	false	✗ no
15	false	✗ no
22	true	✓ yes
7	false	✗ no

```
console.log(evens); // [4, 22]
```

4.5 Real-world example 1 — search/filter box on an e-commerce site

Every "filter by price", "filter by category", or search-as-you-type feature you've ever used on Amazon/Flipkart is built on `.filter()`.

```
const products = [
  { name: "Laptop", price: 55000, category: "Electronics" },
  { name: "Shoes", price: 2500, category: "Fashion" },
  { name: "Phone", price: 18000, category: "Electronics" }
];

const electronicsUnder20k = products.filter(
  p => p.category === "Electronics" && p.price < 20000
);
console.log(electronicsUnder20k);
// [{ name: "Phone", price: 18000, category: "Electronics" }]
```

4.6 Real-world example 2 — removing invalid/empty entries from a form

```
const emailInputs = ["a@test.com", "", " ", "b@test.com", null];
const validEmails = emailInputs.filter(email => email && email.trim().length > 0);
console.log(validEmails); // ["a@test.com", "b@test.com"]
```

Explain `email && email.trim().length > 0` : first checks the value isn't `null / undefined` /empty-string (all falsy), THEN safely calls `.trim()` only if it passed that first check — this is "short-circuit evaluation," worth a 1-minute detour.

4.7 Real-world example 3 — deleting an item from a to-do list (React state update)

"Delete" in a React to-do app is almost never actually deleting from an array in place — it's creating a NEW array that excludes the one item, because React state must be treated as immutable.

```
function deleteTodo(todos, idToRemove) {
  return todos.filter(todo => todo.id !== idToRemove);
}

const todos = [{id:1, text:"Buy milk"}, {id:2, text:"Walk dog"}];
const updated = deleteTodo(todos, 1);
console.log(updated); // [{id:2, text:"Walk dog"}]
```

4.8 filter() returning an empty array — not an error!

```
const nums = [1, 3, 5];
const evens = nums.filter(n => n % 2 === 0);
console.log(evens); // [] — an empty array, NOT undefined, NOT an error. Perfectly valid.
```

Beginners sometimes panic when filter returns `[]`, thinking something broke. Teach: "An empty array is a completely valid, successful result — it just means nothing matched."

5. reduce() — Collapse an Array Into One Thing

5.1 What it does, in one sentence

`reduce()` walks through the array carrying a running "accumulated" value forward from each step to the next, and at the end gives you that single final accumulated value (a number, string, object, or even a new array).

5.2 The snowball analogy (use this in class — it works)

Imagine rolling a snowball down a hill covered in patches of snow (the array elements). At each patch, the snowball (the **accumulator**) picks up that patch's snow and gets a little bigger. By the time it reaches the bottom, one single snowball (the final result) represents everything it picked up along the way. `reduce()` is exactly this: one running value, updated once per element, ending in a single final value.

5.3 Syntax — this is the one with the scary-looking signature

```
const result = array.reduce((accumulator, currentElement, index, array) => {
  return updatedAccumulator; // what the accumulator becomes for the NEXT step
}, initialValue); // <-- starting value of the accumulator
```

Two arguments to `.reduce()` itself: (1) the callback function, (2) the **initial value** of the accumulator. The callback's first parameter, `accumulator`, is the running value; the second, `currentElement`, is like `map/filter`'s usual element.

5.4 Building it from first principles

```
function myReduce(arr, callback, initialValue) {
  let accumulator = initialValue;
  for (let i = 0; i < arr.length; i++) {
    accumulator = callback(accumulator, arr[i], i, arr); // overwrite accumulator each time
  }
  return accumulator; // just ONE final value, not an array
}
```

5.5 Step-by-step trace (sum of an array) — walk through this SLOWLY

```
const nums = [10, 20, 30];
const total = nums.reduce((acc, curr) => acc + curr, 0);
```

Step	acc (incoming)	curr	acc + curr	acc (outgoing, used in next step)
Start	—	—	—	0 (the initial value we gave)
1	0	10	0+10=10	10
2	10	20	10+20=30	30
3	30	30	30+30=60	60

```
console.log(total); // 60
```

Why the initial value matters: If you omit it, `reduce` uses the array's FIRST element as the starting accumulator and starts looping from the SECOND element instead. This works for sums but silently breaks the moment your accumulator needs to be a different shape than an array element (e.g. accumulating into an object or a count) — and it throws a `TypeError` on an empty array. **Teach students to ALWAYS pass an initial value explicitly.** It's one of the most common source of "reduce doesn't work" bugs.

5.6 Real-world example 1 — cart total (the classic POS/e-commerce use case)

Any "cart total" you've seen on a shopping site is a reduce.

```
const cart = [
  { name: "Pen", price: 10, qty: 3 },
  { name: "Notebook", price: 50, qty: 2 },
  { name: "Bag", price: 700, qty: 1 }
];

const cartTotal = cart.reduce((acc, item) => acc + item.price * item.qty, 0);
console.log(cartTotal); // 30 + 100 + 700 = 830
```

5.7 Real-world example 2 — counting occurrences (accumulator as an object)

Reduce doesn't have to accumulate a number — the accumulator can be an object, which is exactly how you'd build a "votes per candidate" or "orders per city" summary.

```
const votes = ["Amit", "Riya", "Amit", "Zara", "Riya", "Amit"];

const voteCount = votes.reduce((acc, name) => {
  acc[name] = (acc[name] || 0) + 1; // if not seen before, start at 0, then +1
  return acc; // MUST return the accumulator every time!
}, {});

console.log(voteCount); // { Amit: 3, Riya: 2, Zara: 1 }
```

The #1 reduce mistake: Forgetting to `return acc` inside the callback. If you forget, the accumulator becomes `undefined` starting from the second iteration, and everything downstream breaks or errors.

5.8 Real-world example 3 — grouping data (very common interview + real task)

```
const students = [
  { name: "Aarav", grade: "A" },
  { name: "Priya", grade: "B" },
  { name: "Zara", grade: "A" },
  { name: "Rohan", grade: "C" }
];

const groupedByGrade = students.reduce((acc, student) => {
  if (!acc[student.grade]) acc[student.grade] = [];
  acc[student.grade].push(student.name);
  return acc;
}, {});

console.log(groupedByGrade);
// { A: ["Aarav", "Zara"], B: ["Priya"], C: ["Rohan"] }
```

5.9 Mind-blowing fact: reduce can replace map AND filter

Because reduce is the most general-purpose of all array methods, you can technically implement `map` and `filter` using only `reduce`. This is a favourite senior-level interview question: "Implement map using reduce."

```
// map, using reduce
const double = [1, 2, 3].reduce((acc, curr) => {
  acc.push(curr * 2);
  return acc;
}, []);
console.log(double); // [2, 4, 6]

// filter, using reduce
const evensOnly = [1, 2, 3, 4].reduce((acc, curr) => {
  if (curr % 2 === 0) acc.push(curr);
  return acc;
}, []);
console.log(evensOnly); // [2, 4]
```

6. find() & findIndex() — Get One Matching Element

6.1 What it does, in one sentence

`find()` returns the **first element** for which your predicate returns `true` — and stops looking immediately once it finds it. If nothing matches, it returns `undefined`.

6.2 Syntax

```
const result = array.find((element, index, array) => {
  return true or false;
});
```

6.3 find() vs filter() — the key distinction

	find()	filter()
Returns	The single matching ELEMENT (or <code>undefined</code>)	An ARRAY of all matching elements (possibly empty)
Stops early?	Yes — stops at the first match (more efficient for "does this exist" checks)	No — always checks every element
Use when	You expect (or only want) ONE result, e.g. finding a user by unique ID	You want ALL matches

6.4 Real-world example — finding a user by ID (extremely common)

```
const users = [
  { id: 1, name: "Aarav" },
  { id: 2, name: "Priya" },
  { id: 3, name: "Zara" }
];

const user = users.find(u => u.id === 2);
console.log(user); // { id: 2, name: "Priya" }

const missing = users.find(u => u.id === 99);
console.log(missing); // undefined
```

Always guard against undefined: Since `find` can return `undefined`, immediately accessing a property on the result (`user.name`) will throw `"Cannot read properties of undefined"` if there was no match. Always check first, or use optional chaining: `user?.name`.

6.5 findIndex() — same idea, but returns the position

```
const idx = users.findIndex(u => u.id === 2);
console.log(idx); // 1 (Priya is at index 1)

const notFoundIdx = users.findIndex(u => u.id === 99);
console.log(notFoundIdx); // -1 (this is the "not found" signal, not undefined)
```

Real use case: You need the index (not the object) when you're about to update or remove that exact item in place — e.g. `users.splice(idx, 1)` to remove a user, which needs a numeric position, not the object itself.

7. some() & every() — Ask a Yes/No Question About the Array

7.1 some() — "is there AT LEAST ONE?"

`some()` returns `true` if AT LEAST ONE element satisfies the condition, otherwise `false`. It stops checking as soon as it finds one true case.

```
const ages = [15, 22, 8, 30];
const hasAdult = ages.some(age => age >= 18);
console.log(hasAdult); // true (22 and 30 qualify)
```

7.2 every() — "do ALL of them satisfy this?"

`every()` returns `true` only if ALL elements satisfy the condition. It stops and returns `false` the moment it finds ONE that fails.

```
const ages = [15, 22, 8, 30];
const allAdults = ages.every(age => age >= 18);
console.log(allAdults); // false (15 and 8 fail)
```

7.3 Real-world example — form validation

"Enable the Submit button only when every required field is filled" and "show a warning if at least one field has an error" are the textbook uses of `every()` and `some()` respectively.

```
const formFields = [
  { name: "email", value: "a@test.com" },
  { name: "password", value: "" },
  { name: "phone", value: "9876543210" }
];

const allFilled = formFields.every(field => field.value.trim() !== "");
console.log(allFilled); // false – password is empty, so Submit stays disabled

const hasAtLeastOneError = formFields.some(field => field.value.trim() === "");
console.log(hasAtLeastOneError); // true – show a validation warning
```

7.4 Real-world example — permission/role checks

```
const userRoles = ["editor", "viewer"];
const requiredRoles = ["admin", "editor"];

const hasAccess = requiredRoles.some(role => userRoles.includes(role));
console.log(hasAccess); // true, because "editor" is in both
```

7.5 Edge case that trips people up: empty arrays

```
console.log([].some(x => x > 5)); // false – nothing to satisfy the condition
console.log([].every(x => x > 5)); // true – "vacuous truth": there's no element to disprove it!
```

This `every()` result surprises almost every student the first time. Explain it as: "every() is only lying if it can point to ONE bad example. With an empty array, it can never point to a bad example, so it defaults to true." This is a genuinely popular tricky interview question — see Section 14.

8. Rest Parameter — Collecting Loose Values

Important framing for the class: Rest and spread are NOT array methods — they're syntax (using the same `...` three-dot symbol) that do almost opposite jobs. Teach them right after array methods because students constantly encounter both together, but be explicit that this is a topic switch.

8.1 What "rest" means, first principles

The **rest parameter** lets a function accept any number of arguments and "gather the rest of them" into a single real array, instead of you having to know in advance how many arguments will be passed.

8.2 The problem it solves

```
// OLD WAY — before rest parameters, you'd use the array-like "arguments" object
function oldSum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}
console.log(oldSum(1, 2, 3)); // 6
```

The `arguments` object is clunky: it's not a true array (no `.map` / `.filter` on it directly), it doesn't exist in arrow functions, and it's easy to misuse.

8.3 Syntax and how it works

```
function sum(...numbers) { // "gather every argument passed into a REAL array called numbers"
  console.log(numbers);     // a genuine array — .map, .filter, .reduce all work
  return numbers.reduce((acc, n) => acc + n, 0);
}

console.log(sum(1, 2, 3));   // [1, 2, 3] then 6
console.log(sum(10, 20, 30, 40)); // [10, 20, 30, 40] then 100
```

8.4 Rest can gather "everything left over" after named parameters

```
function introduce(firstName, lastName, ...hobbies) {
  console.log(`${firstName} ${lastName} enjoys: ${hobbies.join(", ")}`);
}
introduce("Aarav", "Sharma", "cricket", "coding", "music");
// "Aarav Sharma enjoys: cricket, coding, music"
```

Rule: the rest parameter must be LAST. `function f(...rest, a)` is a syntax error — JavaScript needs to know exactly how many named parameters come first before it can "gather everything else."

8.5 Rest in destructuring (a second use of the same syntax)

```
const scores = [90, 85, 70, 60, 55];
const [topScore, secondScore, ...remainingScores] = scores;
console.log(topScore, secondScore); // 90 85
console.log(remainingScores);       // [70, 60, 55]

const student = { name: "Priya", age: 22, city: "Delhi", course: "MERN" };
const { name, ...otherDetails } = student;
console.log(name);           // "Priya"
console.log(otherDetails);   // { age: 22, city: "Delhi", course: "MERN" }
```

9. Spread Operator — Unpacking Values

9.1 What "spread" means, first principles

The **spread operator** takes an iterable (array, string, or object) and "spreads" — unpacks — its individual elements out, wherever you use it. It's the mirror image of `rest`: `rest` GATHERS many things into one array; `spread` SPREADS one array out into many things.

9.2 Spreading into a function call

```
function add3(a, b, c) {
  return a + b + c;
}
const nums = [5, 10, 15];
console.log(add3(...nums)); // same as add3(5, 10, 15) → 30
```

9.3 Real-world example — copying an array (without mutating the original)

In React, you NEVER mutate state directly. Spread is the standard, everyday way to make a safe copy of an array or object before changing it.

```
const original = [1, 2, 3];
const copy = [...original];
copy.push(4);
console.log(original); // [1, 2, 3]    <-- untouched
console.log(copy);     // [1, 2, 3, 4]
```

Shallow copy warning: Spread only copies one level deep. If the array contains objects, the inner objects are still shared by reference between the original and the copy.

```
const original = [{ id: 1, name: "A" }];
const copy = [...original];
copy[0].name = "CHANGED";
console.log(original[0].name); // "CHANGED" too! Same nested object, shared by reference.
```

9.4 Real-world example — merging arrays

```
const fruits = ["apple", "banana"];
const veggies = ["carrot", "potato"];
const groceries = [...fruits, ...veggies, "milk"];
console.log(groceries); // ["apple", "banana", "carrot", "potato", "milk"]
```

9.5 Real-world example — updating React state immutably (very high-frequency real code)

```
// Adding a new todo without mutating the old array
setTodos(prevTodos => [...prevTodos, newTodo]);

// Updating one field of one object inside an array of objects
setUsers(prevUsers =>
  prevUsers.map(user =>
    user.id === targetId ? { ...user, isActive: true } : user
  )
);
```

This second example is a genuinely important real pattern — spend a moment on it: `{ ...user, isActive: true }` copies all of `user`'s existing fields, then overwrites just `isActive`. Combined with `map` to only touch the ONE matching object.

9.6 Spreading objects

```
const defaults = { theme: "light", fontSize: 14 };
const userPrefs = { fontSize: 18 };
const finalSettings = { ...defaults, ...userPrefs };
console.log(finalSettings); // { theme: "light", fontSize: 18 }
```

Order matters! Later spreads override earlier ones for the same key — this is exactly how you implement "defaults with overrides," extremely common in config objects and component props.

9.7 Spreading a string

```
console.log([..."hello"]); // ["h", "e", "l", "l", "o"]
```

10. Rest vs Spread — Side by Side (the #1 confusion)

Both use the exact same `...` symbol, which is precisely why students confuse them. The trick to tell them apart: **look at WHERE the three dots appear.**

	Rest (...)	Spread (...)
Direction	Many values → gathered INTO one array	One array/object → spread OUT into many values
Where it appears	In a function's PARAMETER list, or on the LEFT side of a destructuring assignment	In a function CALL's arguments, or inside an array/object LITERAL <code>[]</code> / <code>{ }</code>
Example	<code>function f(...args)</code>	<code>f(...myArray)</code>
Simple memory trick	Rest = "the REST of them, please collect" (used when DEFINING/receiving)	Spread = "SPREAD it all out here" (used when GIVING/using)

```
// REST – gathering (appears in a parameter list)
function total(...prices) {
  return prices.reduce((a, b) => a + b, 0);
}

// SPREAD – unpacking (appears in a call/literal)
const priceList = [100, 200, 300];
console.log(total(...priceList)); // 600
```

Interview one-liner students should be able to say: "Rest collects multiple elements into an array; spread expands an array into multiple elements. Same syntax, opposite direction, and you tell them apart by context — parameter list vs. literal/call."

11. Chaining Methods — Real Mini-Projects

Because `map` and `filter` both return arrays, you can chain them directly, one after another, to express a multi-step data pipeline in a single readable line. This is where these methods show their real power over manual loops.

11.1 Mini project — "Get the names of all in-stock products under ₹1000, in uppercase"

```
const products = [
  { name: "Pen", price: 10, inStock: true },
  { name: "Watch", price: 1500, inStock: true },
  { name: "Notebook", price: 50, inStock: false },
  { name: "Bag", price: 700, inStock: true }
];

const result = products
  .filter(p => p.inStock && p.price < 1000) // Step 1: keep matching products
  .map(p => p.name.toUpperCase());         // Step 2: transform to names, uppercase

console.log(result); // ["PEN", "BAG"]
```

Walk through this out loud: "First we filter down to the products we care about — a SHORTER array of the same kind of objects. THEN we map that shorter array into the shape we actually want to display."

11.2 Mini project — "Total revenue from only completed orders"

```
const orders = [
  { id: 1, amount: 500, status: "completed" },
  { id: 2, amount: 300, status: "cancelled" },
  { id: 3, amount: 750, status: "completed" },
  { id: 4, amount: 200, status: "pending" }
];

const completedRevenue = orders
  .filter(order => order.status === "completed")
  .reduce((sum, order) => sum + order.amount, 0);

console.log(completedRevenue); // 500 + 750 = 1250
```

11.3 Mini project — "Build a student report card summary"

```
const students = [
  { name: "Aarav", marks: [80, 90, 70] },
  { name: "Priya", marks: [95, 85, 92] },
  { name: "Zara", marks: [40, 55, 60] }
];

const report = students.map(student => {
  const total = student.marks.reduce((a, b) => a + b, 0);
  const average = total / student.marks.length;
  return {
    name: student.name,
    average: average.toFixed(2),
    passed: student.marks.every(mark => mark >= 40)
  };
});

console.log(report);
/* [
  { name: "Aarav", average: "80.00", passed: true },
  { name: "Priya", average: "90.67", passed: true },
  { name: "Zara", average: "51.67", passed: true }
] */
```

This example is great for class because it uses FOUR concepts in one problem: `map` (outer transform), `reduce` (sum marks), a plain calculation (average), and `every` (pass/fail check) — all nested naturally, the way real code actually reads.

11.4 Mini project — live search with debounced filter (React, ties back to real UI)

```
function ProductSearch({ products }) {
  const [query, setQuery] = useState("");

  const filteredProducts = products.filter(p =>
    p.name.toLowerCase().includes(query.toLowerCase())
  );

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      <ul>
        {filteredProducts.map(p => <li key={p.id}>{p.name}</li>)}
      </ul>
    </div>
  );
}
```

```
    </ul>
  </div>
);
}
```

Point out: this single component uses filter (search logic) AND map (rendering the list) together — the two most-used methods in any real React app, working side by side.

12. Comparison Cheat-Sheet — map vs forEach vs filter vs reduce

Method	Returns	Original array modified?	Stops early?	Typical job
map()	New array, same length	No	No	Transform every element
forEach()	undefined	No (unless you mutate inside)	No	Just perform an action per element
filter()	New array, ≤ length	No	No	Keep only matching elements
reduce()	Single value (any type)	No	No	Combine everything into one result
find()	One element or undefined	No	Yes	Get the first match
findIndex()	Index or -1	No	Yes	Get the position of the first match
some()	true / false	No	Yes	"Does at least one match?"
every()	true / false	No	Yes	"Do all of them match?"

One sentence that ties the whole chapter together: "Every one of these methods loops through the array and calls your function on each element — they only differ in what they do with the return value." Have students recite this from memory before the exit ticket.

13. Interview Questions (Conceptual)

Q1. What is the difference between `map()` and `forEach()`? Easy

`map()` returns a new array built from the callback's return values and is chainable; `forEach()` always returns undefined and exists purely to perform a side effect per element.

Q2. Does `map()`/`filter()`/`reduce()` mutate the original array? Easy

No. All three return a brand-new array (or value, for `reduce`) and leave the original array completely untouched — unless you deliberately mutate an object/array that's nested inside an element (shallow-copy caveat).

Q3. Why should you always pass an initial value to `reduce()`? Medium

Without one, `reduce` uses the array's first element as the starting accumulator and skips it in the loop — this breaks when the accumulator needs a different shape than an element (e.g. building an object), and throws a `TypeError` on an empty array.

Q4. Implement `map()` using `reduce()`. Medium

```
const double = [1,2,3].reduce((acc, curr) => {
  acc.push(curr * 2);
  return acc;
}, []);
```

This shows `reduce` is the most general-purpose of the three — `map` and `filter` can both be expressed in terms of it.

Q5. What's the difference between `find()` and `filter()`? Easy

`find()` returns the first matching ELEMENT (or undefined) and stops as soon as it's found; `filter()` always scans the whole array and returns an ARRAY of ALL matches (possibly empty).

Q6. What does `every()` return for an empty array? Why? Hard

`true` — this is "vacuous truth." `every()` only returns false when it can point to a specific failing element; with zero elements, it can never find one, so it defaults to `true`.

Q7. What is a higher-order function? Name three built-in examples. Medium

A function that accepts another function as an argument or returns a function. `map`, `filter`, and `reduce` all qualify because each accepts a callback.

Q8. Difference between rest and spread — they look identical (...)! Medium

Rest COLLECTS multiple values into a single array (used in function parameters or the left side of destructuring). Spread EXPANDS a single array/object into multiple values (used in function calls or array/object literals). Same syntax, opposite direction — tell them apart from context.

Q9. Why does spread only make a "shallow" copy, and why does that matter? Hard

Spread copies references one level deep. If an array/object contains nested objects, those inner objects are still shared between the original and the copy — mutating a nested object through the copy will also affect the original. Matters hugely in React, where you must never accidentally mutate shared state.

Q10. How would you remove duplicate values from an array using array methods (or another built-in)? Medium

```
const unique = arr.filter((val, idx) => arr.indexOf(val) === idx);
// or, simpler using Set + spread:
const unique2 = [...new Set(arr)];
```

Q11. Can you break out of a `map()`/`forEach()` loop early with `break`? Hard

No — `break`/`continue` don't work inside these callback-based methods because each call to the callback is a completely separate function invocation, not part of one physical loop body. If you need early exit, use a plain for loop, or use some()/every() (which DO stop early) as a workaround, or find().`

Q12. What does `.map()` do if the callback doesn't return anything? Medium

Every element in the resulting array becomes undefined, because a function with no explicit return statement implicitly returns undefined. Very common bug source with arrow functions that use curly braces but forget the `return` keyword.`

Q13. Can `filter()` and `map()` be replaced by a single `reduce()` call for efficiency? Hard

Yes — chaining `.filter().map()` loops the array twice; a single `.reduce()` that both checks the condition and transforms in one pass loops it once. In performance-critical code (very large arrays) this matters; for typical UI-scale data, readability of the chained version usually wins.

14. Tricky Output-Based Questions

Read each snippet WITHOUT running it, predict the output, then verify. This is exactly the format interviewers use to test real understanding versus memorized syntax.

Q1. Easy

```
const arr = [1, 2, 3];
const result = arr.map(n => {
  n * 2;    // <-- no "return"!
});
console.log(result);
```

Output: `[undefined, undefined, undefined]` — curly-brace arrow function bodies need an explicit `return`; without it, each call implicitly returns `undefined`.

Q2. Medium

```
const nums = [1, 2, 3, 4, 5];
const output = nums.filter(n => n > 10);
console.log(output);
console.log(output.length);
```

Output: `[]` then `0` — an empty array is a valid result, not an error, not `undefined`.

Q3. Hard

```
const total = [1, 2, 3].reduce((acc, curr) => acc + curr);
console.log(total);
const total2 = [].reduce((acc, curr) => acc + curr);
console.log(total2);
```

First line: `6` — with no initial value, `reduce` uses index 0 (value 1) as the starting accumulator and loops from index 1. Second line: **throws a `TypeError`** — "Reduce of empty array with no initial value." This is exactly why we always teach passing an initial value.

Q4. Hard

```
console.log([].every(x => x > 100));
console.log([].some(x => x > 100));
```

Output: `true` then `false` — `every()` vacuously returns `true` on an empty array (it can't find a counter-example); `some()` returns `false` because there's nothing to satisfy the condition.

Q5. Medium

```
const original = [1, 2, 3];
const doubled = original.map(n => n * 2);
original.push(4);
console.log(original);
console.log(doubled);
```

Output: `[1, 2, 3, 4]` then `[2, 4, 6]` — `map` created a completely separate array at the time it ran; later mutating the original has zero effect on the already-returned `doubled` array.

Q6. Hard

```
const arr = [{ id: 1, name: "A" }, { id: 2, name: "B" }];
const copy = [...arr];
copy[0].name = "CHANGED";
console.log(arr[0].name);
```

Output: `"CHANGED"` — spread is a SHALLOW copy. The outer array is new, but the objects inside are still the exact same objects in memory, shared by both `arr` and `copy`.

Q7. Medium

```
function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
console.log(sum(1, 2, 3));
console.log(sum());
```

Output: `6` then `0` — with zero arguments, `nums` is simply an empty array `[]`, and `reduce` WITH an explicit initial value of 0 safely returns 0 (no `TypeError`, unlike Q3's second line).

Q8. Hard

```
const users = [{ id: 1 }, { id: 2 }, { id: 3 }];
const found = users.find(u => u.id === 5);
console.log(found);
console.log(found.id);
```

Output: `undefined`, then **throws a TypeError**: "Cannot read properties of undefined (reading 'id')" — a classic real-world bug from not guarding a `find()` result before using it.

Q9. Hard

```
const a = [1, [2, 3], [4, [5, 6]]];
console.log([...a]);
console.log(a.flat());
console.log(a.flat(Infinity));
```

Output: `[1, [2, 3], [4, [5, 6]]]` (spread only unpacks ONE level, and here that level is already the outer array, so it just shallow-copies — nested arrays stay nested); `[1, 2, 3, 4, [5, 6]]` (`flat()` defaults to flattening 1 level deep); `[1, 2, 3, 4, 5, 6]` (`Infinity` flattens fully, regardless of depth). Good example that spread and `flat()` are NOT the same tool for nested arrays.

Q10. Medium

```
const nums = [1, 2, 3];
console.log(nums.map(String));
```

Output: `["1", "2", "3"]` — you can pass a built-in function directly as the callback if its signature happens to line up; `String(1)` returns `"1"`. A nice one to show that the callback doesn't have to be an inline arrow function.

Q11. Hard

```
const arr = [10, 20, 30];
arr.forEach((n, i) => {
  arr[i] = n * 2;
});
console.log(arr);
```

Output: `[20, 40, 60]` — `forEach` itself never mutates the array, but nothing STOPS your callback from mutating it manually via index assignment. This is technically legal but considered bad practice; `map()` is the correct tool when you actually want a transformed array.

Q12. Hard

```
const obj1 = { a: 1, b: 2 };
const obj2 = { b: 99, c: 3 };
console.log({ ...obj1, ...obj2 });
console.log({ ...obj2, ...obj1 });
```

Output: `{ a: 1, b: 99, c: 3 }` then `{ b: 2, c: 3, a: 1 }` — when spreading multiple objects with overlapping keys, whichever spread comes LAST wins for that key. Order matters.

15. Practice Problems for Students (Homework Set)

Give these as homework after class. Difficulty increases gradually; the last two require chaining multiple concepts from this booklet.

1. **Easy** Given an array of numbers, return a new array with each number squared, using `map()`.
2. **Easy** Given an array of strings, return only the strings longer than 5 characters, using `filter()`.
3. **Easy** Given an array of numbers, find the sum and the product of all elements using two separate `reduce()` calls.
4. **Medium** Given an array of employee objects `{name, department, salary}`, use `filter()` and `map()` together to get the names of all employees in the "Engineering" department earning above ₹50,000.
5. **Medium** Given an array of numbers, use `reduce()` to find both the minimum AND maximum in a single pass, returning `{min, max}`.
6. **Medium** Write a function `hasDuplicates(arr)` that uses `some()` to return true if any value appears more than once.
7. **Medium** Write a function using rest parameters that accepts any number of arrays and merges them into a single flat array using spread.
8. **Hard** Given an array of orders `{customer, amount, status}`, produce an object that groups the TOTAL amount spent per customer, counting only orders with `status: "completed"`. (Hint: filter, then reduce into an object.)
9. **Hard** Implement your own version of `Array.prototype.myFilter` as a method added onto the array prototype, using only a plain for loop internally (no built-in filter/map/reduce allowed).
10. **Hard** Given a list of student objects with a `marks` array each, write one chained expression that returns the names of students (in uppercase) whose average mark is above 75, sorted by average descending. (Combines map, filter, reduce, and a sort — a genuine "everything you learned today" capstone problem.)

End of booklet — MyAnatomy Teaching Series. Prepared for classroom use, first-principles style, from zero to interview-ready.